

Optimization with Human Signals

Human-in-the-loop Optimization

Optimizing Interactive Mobility Interfaces with Human Feedback: Multi Objective Bayesian Optimization in Practice

Dr Mark Colley

Warmup: Color Oracle

This game requires two players: Player A (Oracle) and Player B (Optimizer).

Player A thinks about **the two** most beautiful colors.

Player B's task is to figure out what these colors are in eight questions. Only one color can be queried in one question.

When asked about a color, Player A must tell how beautiful it is on a scale from 1 (worst) to 5 (best). After the eighth question, Player B must propose which two colors are Player A's favorites.

Optional: Hands-on to follow up on the Oracle

Google Colab:

https://colab.research.google.com/drive/1QgTCZKiPtJfw1DRkwT96mkvjv9_11eUS



This game requires two players: Player A (Oracle) and Player B (Optimizer).

Player A thinks about **the most** beautiful color.

Player B's task is to figure out what this color is in **five questions**. Only one color can be queried in one question.

When asked about a color, Player A must tell how beautiful it is on a scale from 1 (worst) to 5 (best). After the fifth question, Player B must propose which color is Player A's favorite.

Figuring out the design that is the best match for a person is an HCI problem

A. Game Settings



B. Layout of an App



C. Robot Behavior



Figuring out the design that is the best match for a person is an HCI problem

A. Game Settings



B. Layout of an App



C. Robot Behavior



Design: a set of parameters that describes the system.

Fundamentally, there are two approaches to achieving this best match:

Adaptable

The alteration is done **explicitly** by the **final user**.

Adaptive

The alteration is done **implicitly** by the **system**

A design is a choice of how the system behaves

We collect every knob we are allowed to turn into a single object — a vector of parameters, a configuration, a policy.

Call one such choice a design x .

Not all designs are possible. Every parameter has bounds, and between parameters there are constraints.

Call the set of all possible designs: X

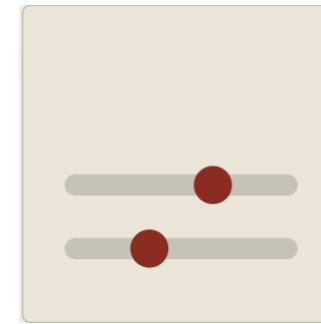
$$x \in X$$

one design x , drawn from the space of all possible designs X

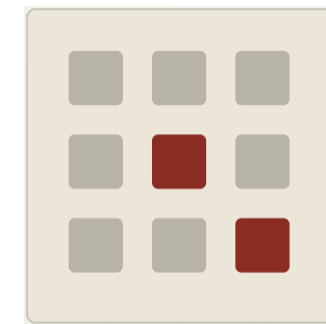
Design Space: the set of all possible designs.

How you describe a design decides what's findable.

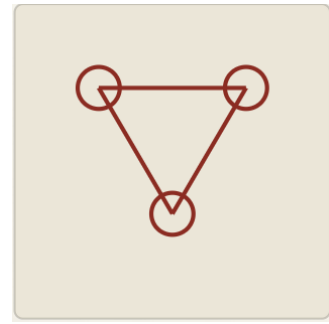
The space X can be continuous, discrete, or structured — and its shape sets the difficulty. A good parameterization makes good designs close together; a bad one hides them.



continuous



discrete



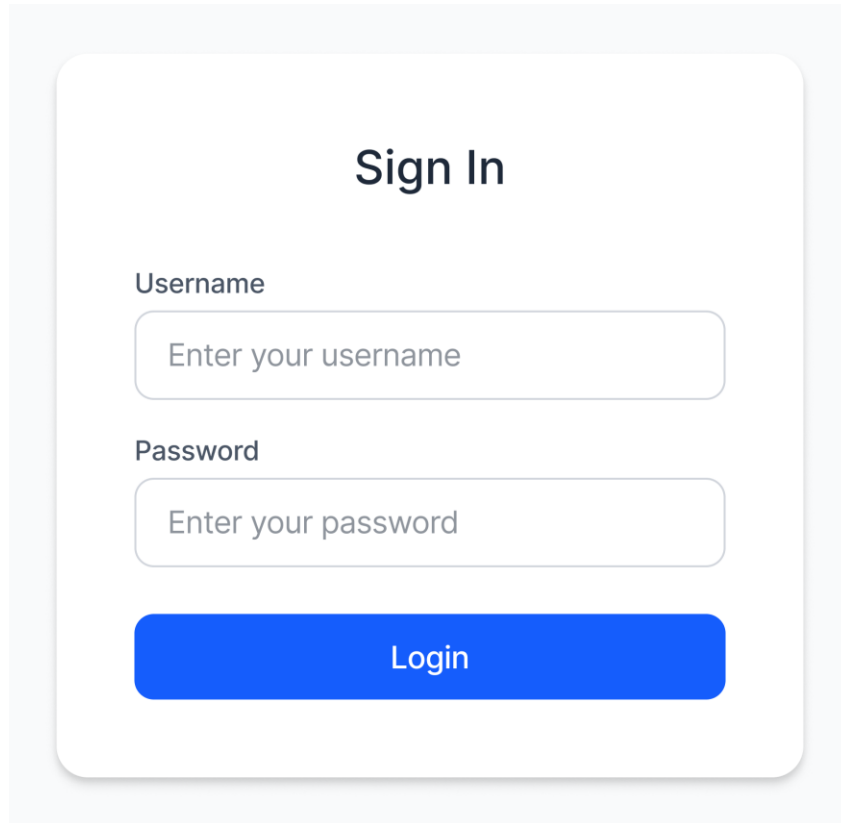
structured

cont. sliders, weights, positions — $\mathbb{R}^d$

disc. menus, on/off, orderings — combinatorial

struct. layouts, programs, graphs — constrained

Example: What could the design space be??



Sign In

Username

Password

Login

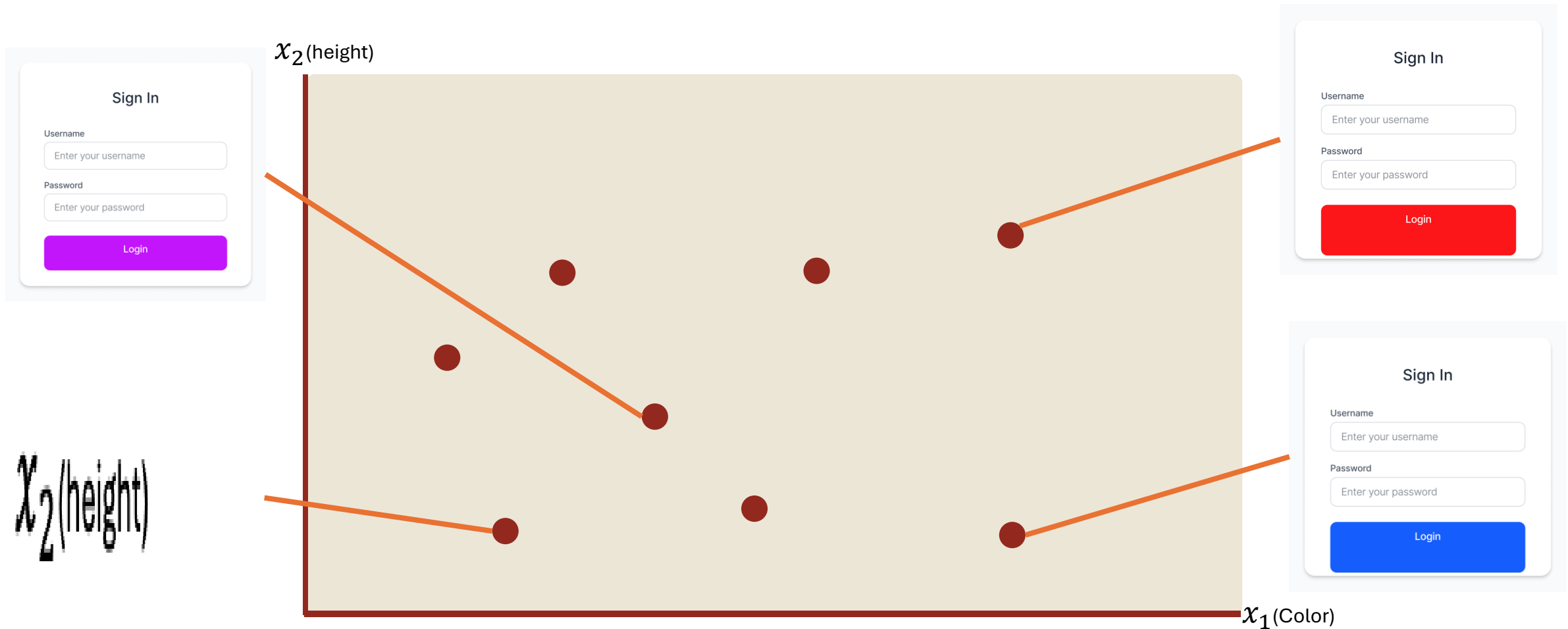
What parameters could be “knobs” on the homepage? What are the bounds? What is the design space then?

Example: parameterize the google home page with two parameters

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

The image shows two identical 'Sign In' forms side-by-side, illustrating a parameterized design. Each form has a title 'Sign In', a 'Username' label, a text input field with placeholder text 'Enter your username', a 'Password' label, a text input field with placeholder text 'Enter your password', and a 'Login' button. The left form has a blue 'Login' button, while the right form has a red 'Login' button. This visualizes how a single design can be parameterized by a color value (e.g., x_2 for the button color).

For any of these, there are many combinations possible.



For any of these, there are many combinations possible.

A handful of continuous knobs, a few discrete choices, and the number of distinct designs explodes past anything we could ever try by hand.

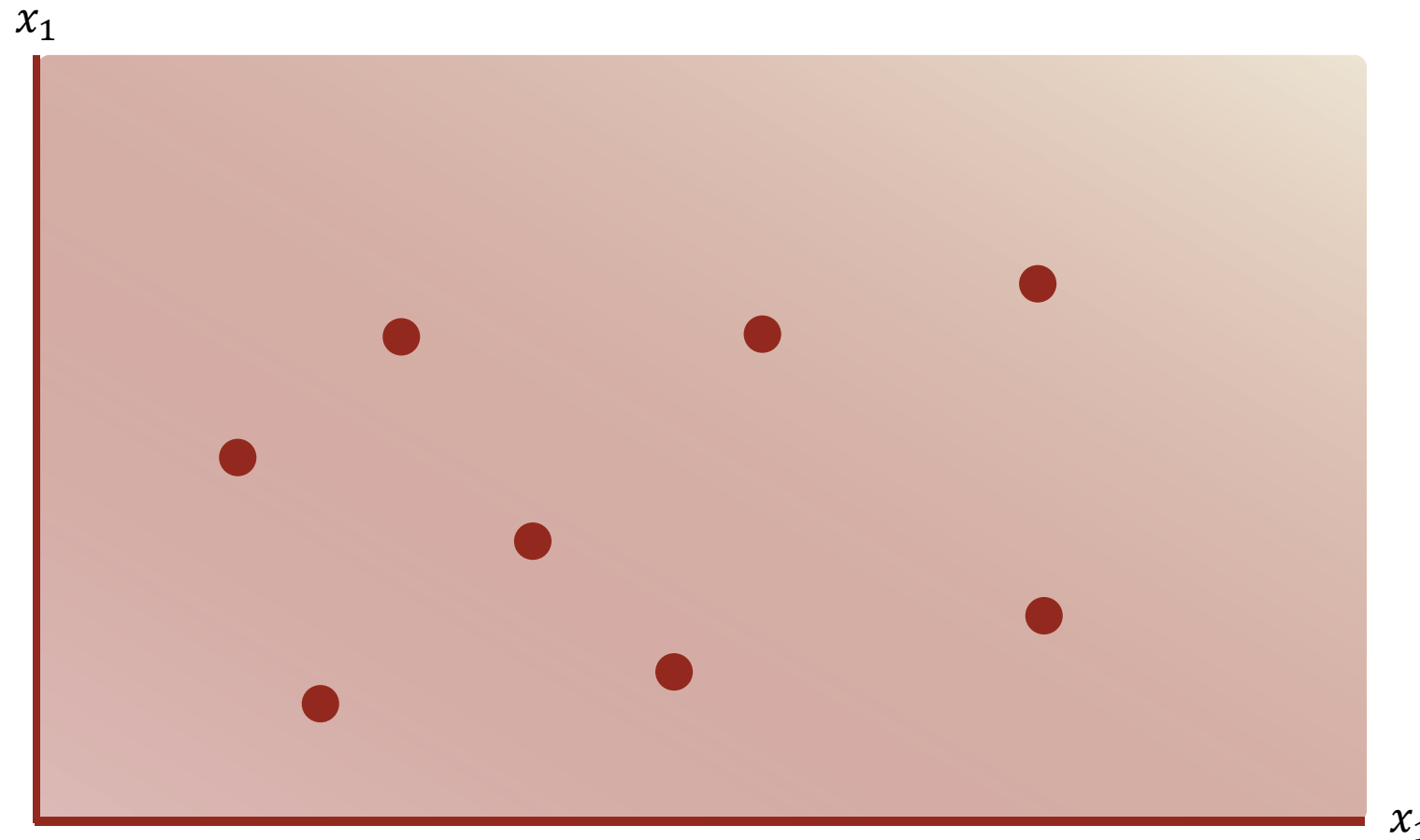
10^8

LAYOUTS FROM 8
BINARY SLOTS

∞

CONTINUOUS
PARAMETERS

For any of these, there are many combinations possible.



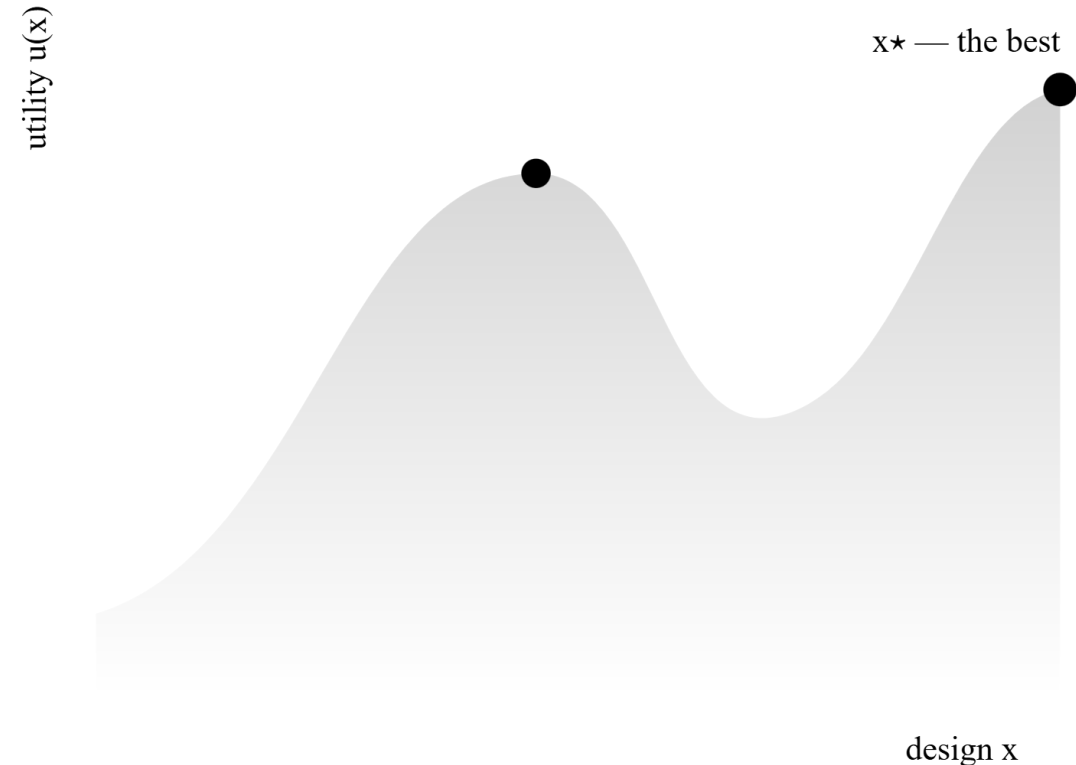
Out of all the possibilities, how do we find the one that works best?

What makes one design better than another?

Imagine a number attached to every design that says how good it is *for this user*: how much they like it, how well it fits, how well it works for them. Call it the design's **utility**.

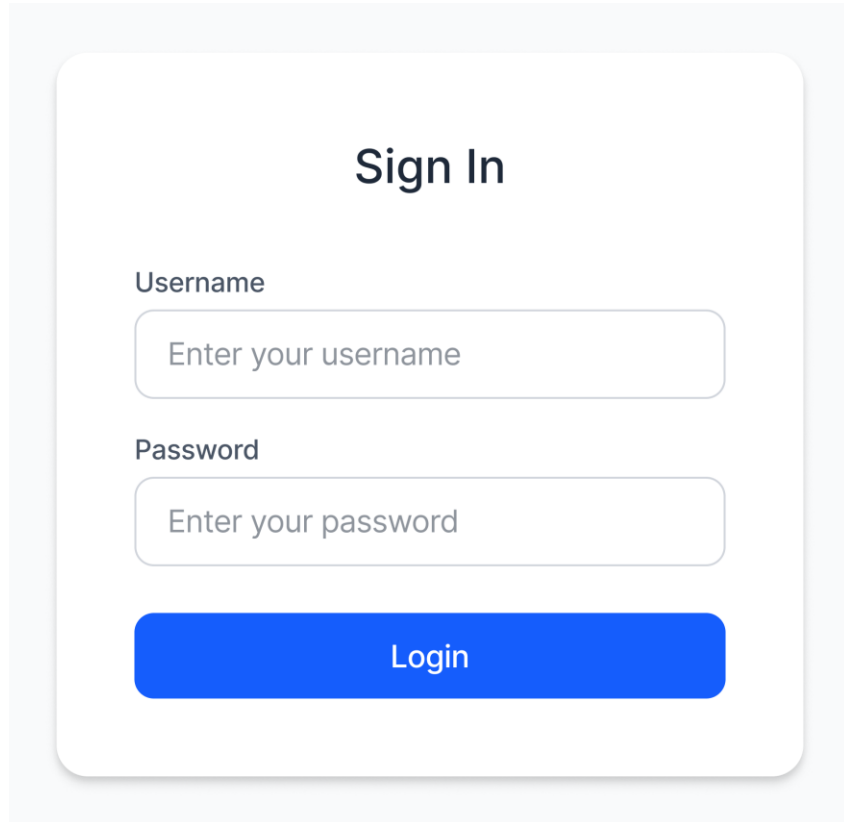
$$u: X \rightarrow \mathbb{R}$$

a function scoring every design — higher is better



“find the best design” has a precise meaning: find the x where u is largest.

Example: What could the utility be?

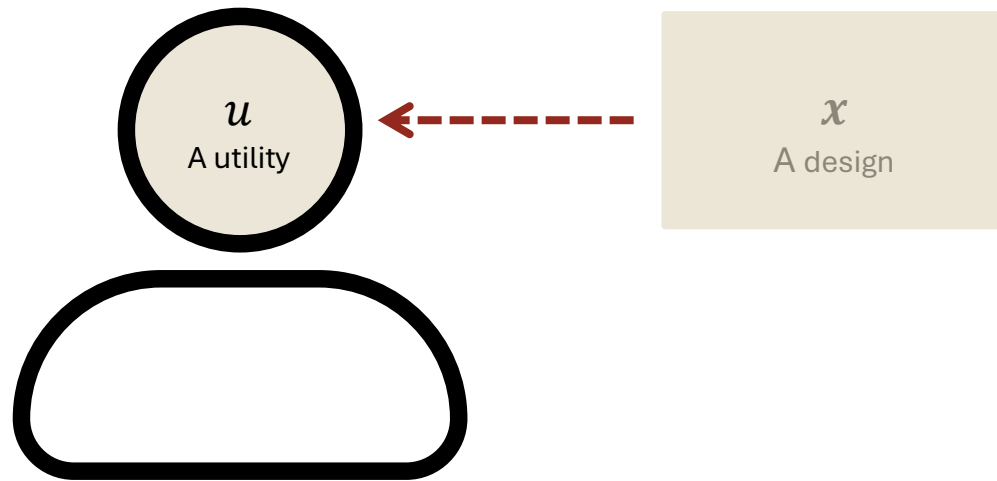


A sign-in form with a white background and rounded corners, set against a light gray background. The form contains the following elements:

- Sign In**: A title centered at the top of the form.
- Username**: A label above a text input field with the placeholder text "Enter your username".
- Password**: A label above a text input field with the placeholder text "Enter your password".
- Login**: A blue button with white text centered below the password field.

What would make a sign in page, a good sign in page? How would you measure that?

The problem: the utility is latent and can only be accessed indirectly.



The utility cannot be expressed:

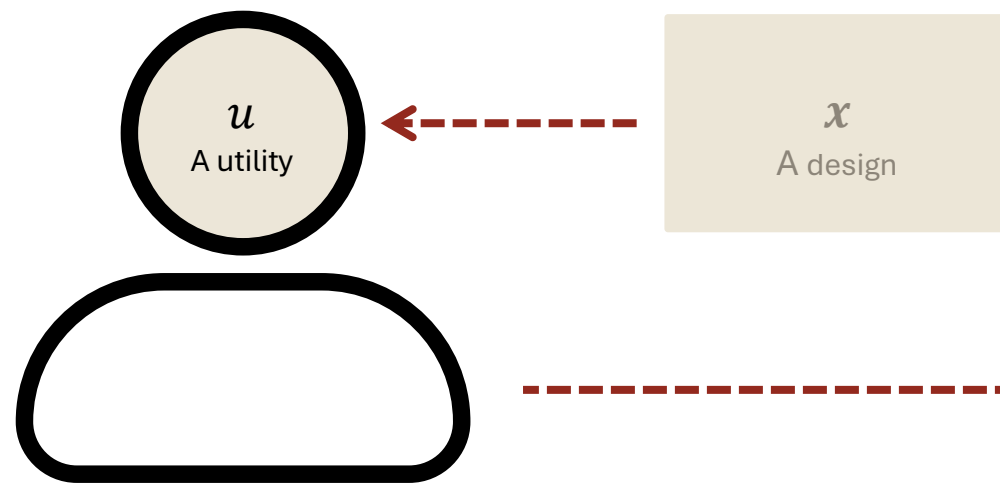
E.g., exact description

It cannot be calculated closed-form:

E.g., performance in a game.

Latent: present but needing particular conditions to become active, obvious, or completely developed

The problem: the utility is latent and can only be accessed indirectly.



We need a human action:

Performance:

- Task completion time
- Search time
- Error rate

Ratings:

- Score
- Ranking
- Preference

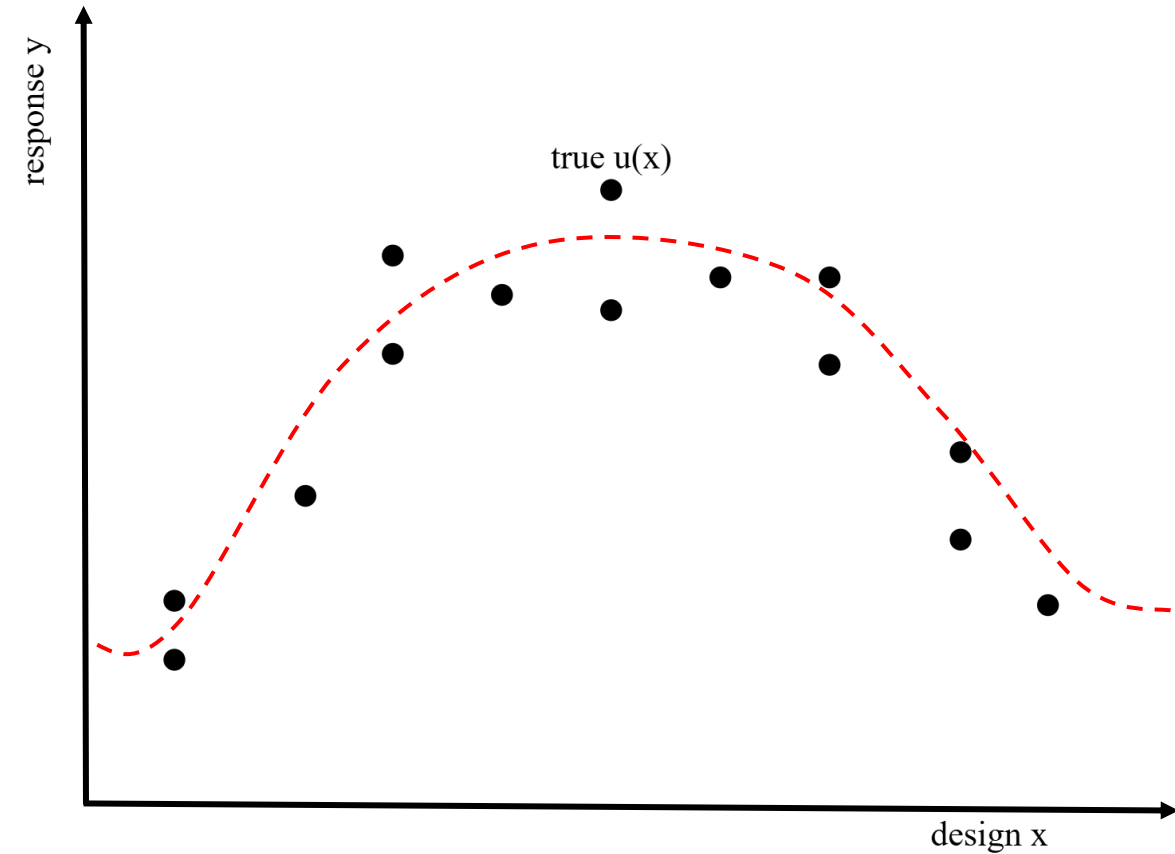
To evaluate a design we need to involve the human

The answers are noisy.

Ask the same person about the same design twice and you may get two different answers. Mood, fatigue, the last thing they saw — all of it perturbs the response.

$$r = u(x) + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma^2)$$

we observe r , a corrupted version of the true utility

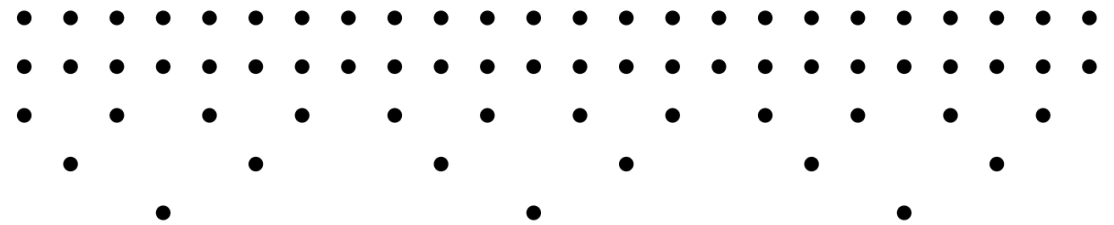


And every answer costs you

Each query spends the user's time, attention and patience. People fatigue, they satisfice, they walk away. Unlike a simulator, you cannot ask a million times.

This is the constraint that defines the field: the budget of queries is tiny — often tens, rarely thousands.

a simulator



millions of cheap evaluations

a person



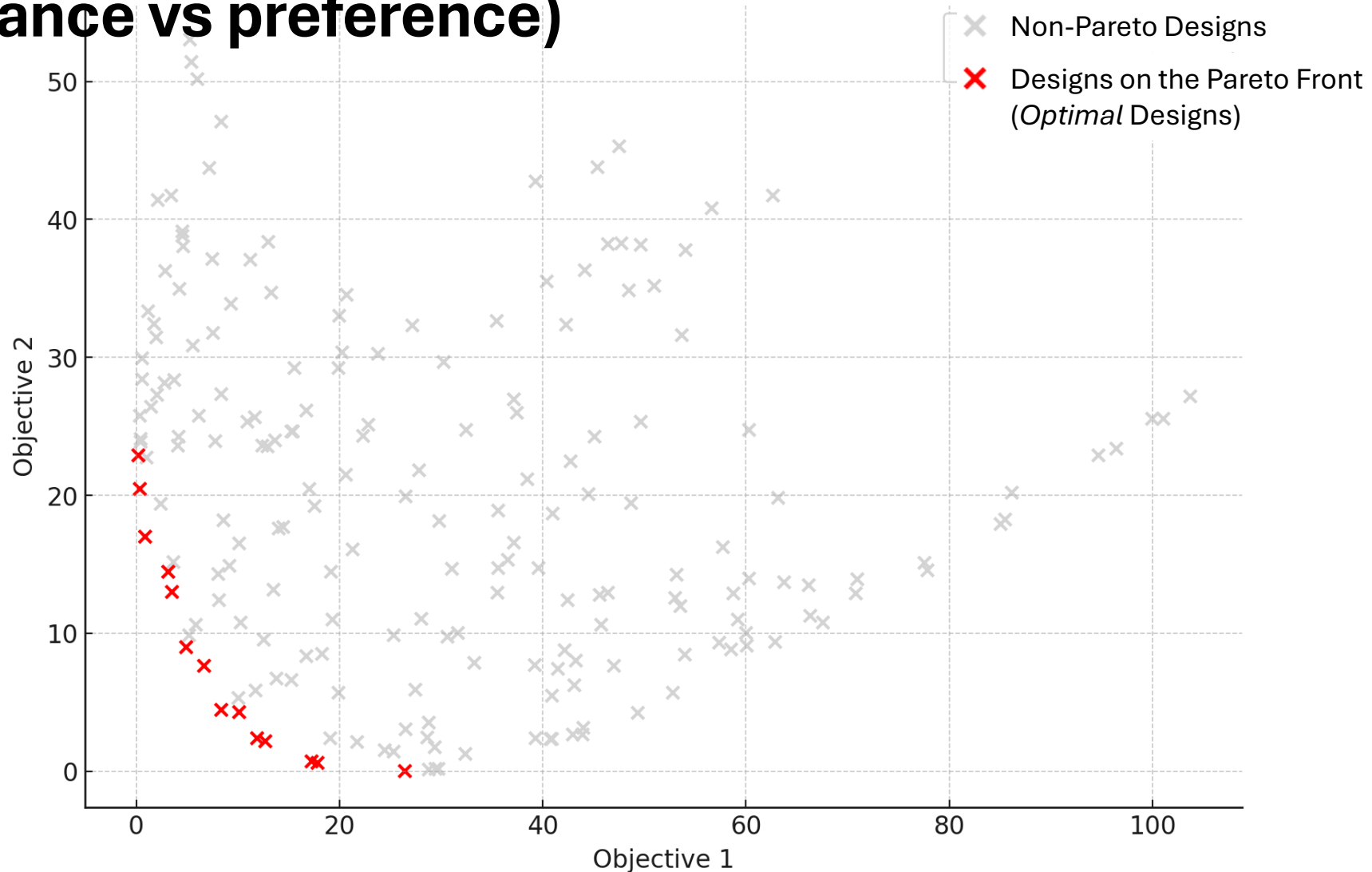
a few dozen, then they're done

Which question we ask shapes what we can learn

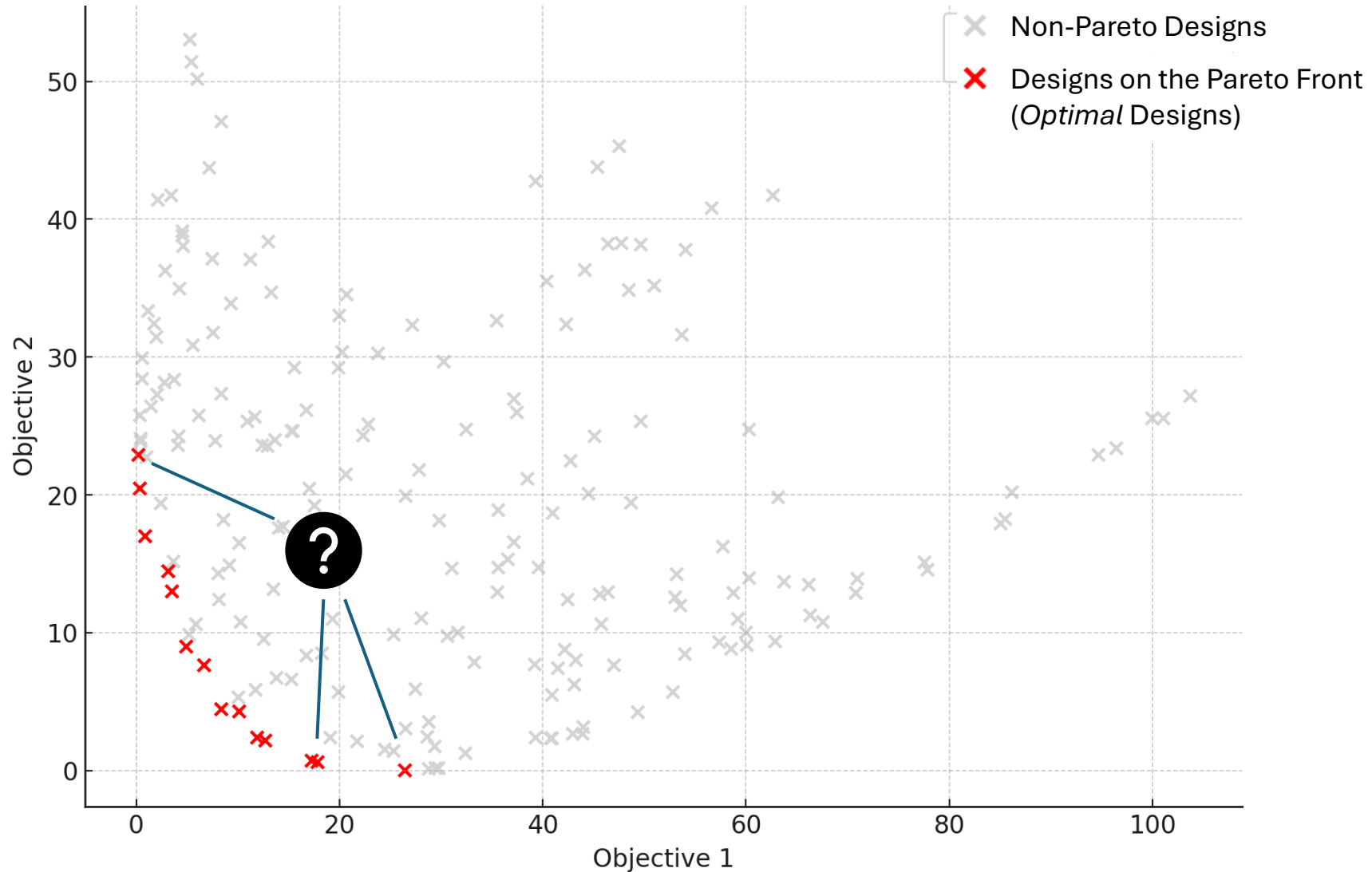
Different query types trade information against effort . Comparisons are easy for people and robust to scale; absolute ratings carry more information but are noisier and drift over time.

QUERY TYPE	WHAT THE USER DOES	SIGNAL	EFFORT
Rating	Scores a design on a scale	Value of $u(x)$, noisy	Low–medium
Pairwise choice	Picks A vs. B	Sign of $u(x_A) - u(x_B)$	Very low
Ranking	Orders a small set	Many comparisons at once	Medium
Demonstration	Shows the desired behaviour	Region of high u	High

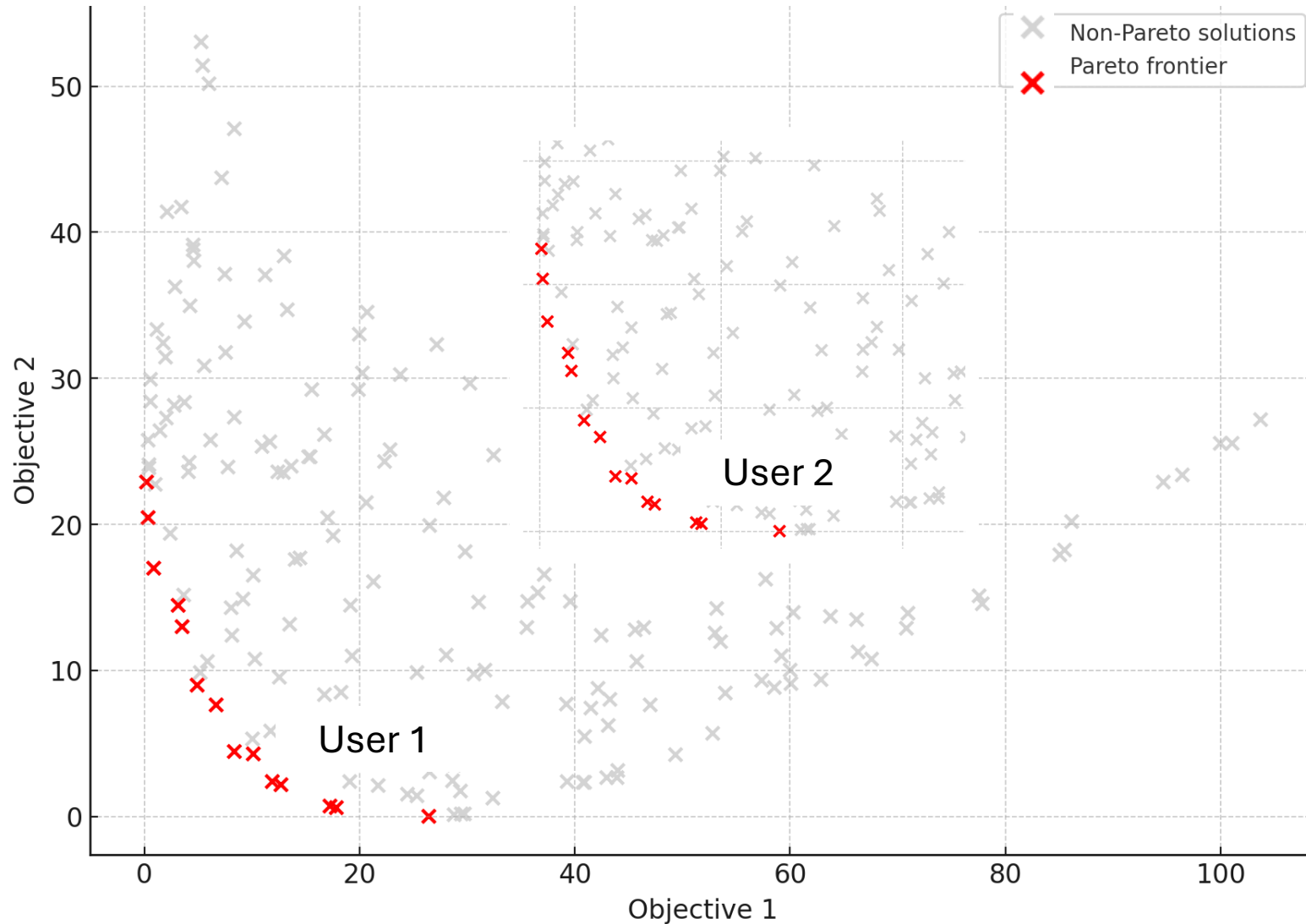
When is a design optimal? There can be trade-offs (performance vs preference)



Which design should I use?



Trade-offs are user specific



With so few queries, where we ask is everything.

Because each query is precious, we don't sample at random. We spend each one where it helps most, balancing exploiting the current best against exploring the unknown.

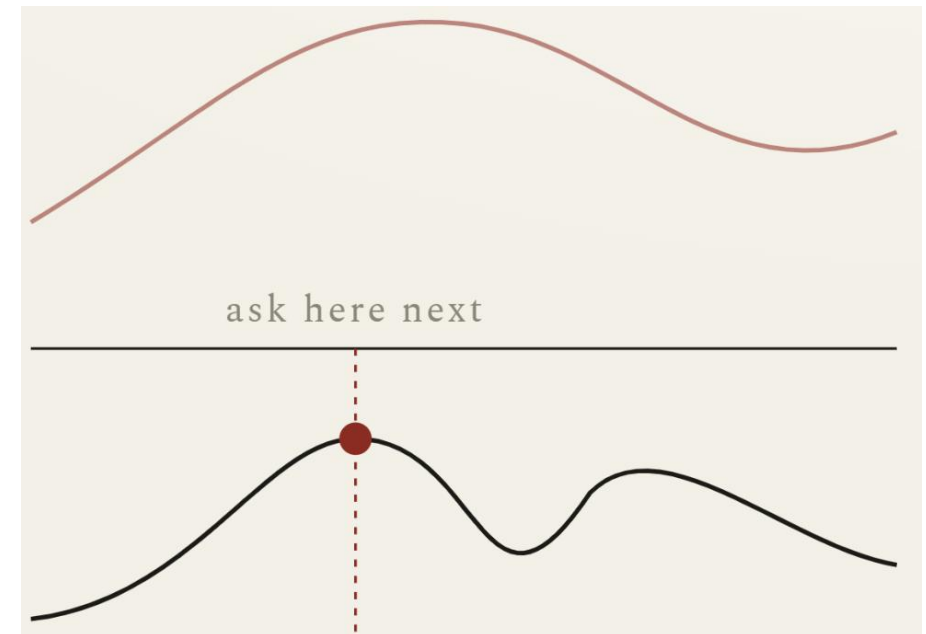
EXPLOIT

ask near the current best to refine it

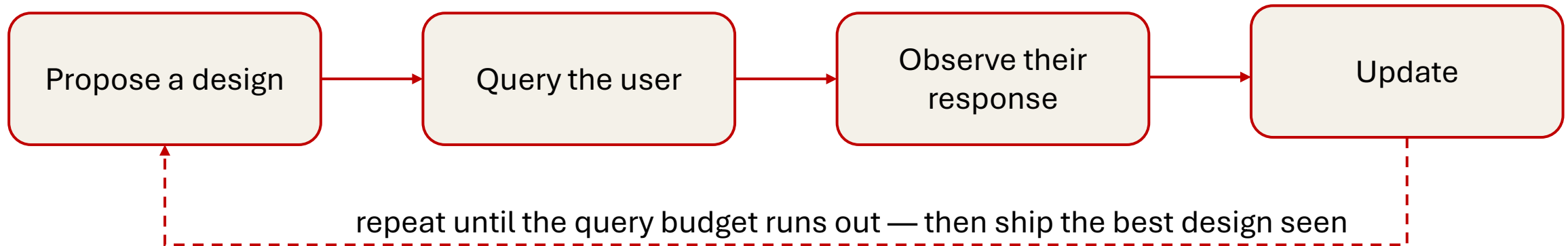
EXPLORE

ask where we're unsure — a better peak may hide

Belief how the person's preference could be shaped



The “Human-in-the-loop” Loop



Definition

Human-in-the-loop (HITL) optimization is the **problem of finding a design** (a set of parameters, a configuration, or a system behavior) that maximizes a **latent user utility** accessible only through **costly, noisy, and indirect queries** to the human.

Unpacking the definition: Design Space

Human-in-the-loop (HITL) optimization is the problem of finding **a design** (a set of parameters, a configuration, or a system behavior) that maximizes a latent user utility accessible only through costly, noisy, and indirect queries to the human.

$$x \in X$$

one design x , drawn from the space of all possible designs X

Unpacking the definition: Design Space

Human-in-the-loop (HITL) optimization is the problem of finding a design (a set of parameters, a configuration, or a system behavior) that maximizes a latent user **utility** accessible only through costly, noisy, and indirect queries to the human.

$$u : X \rightarrow \mathbb{R}$$

a function scoring every design in the design space and mapping it to a real scalar.

Unpacking the definition: Design Space

Human-in-the-loop (HITL) optimization is the problem of finding a design (a set of parameters, a configuration, or a system behavior) that maximizes a latent user utility accessible only through costly, noisy, and indirect **queries** to the human.

$$q \in Q$$

One query drawn from the set of all possible queries

Unpacking the definition: Design Space

Human-in-the-loop (HITL) optimization is the problem of finding a design (a set of parameters, a configuration, or a system behavior) that maximizes a latent user utility accessible only through **costly, noisy, and indirect** queries to the human.

$$r \sim p(r|q; u)$$
$$r = u(x) + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma^2)$$

A response is a corrupted version of the true utility, it depends on the query and utility.

Unpacking the definition: Design Space

Human-in-the-loop (HITL) optimization is the problem of finding a design (a set of parameters, a configuration, or a system behavior) that maximizes a latent user utility accessible only through **costly**, noisy, and indirect queries to the human.

$$c(q) > 0$$

The cost of a query is strictly larger than 0

Unpacking the definition: Design Space

Human-in-the-loop (HITL) optimization is the problem of **finding** a design (a set of parameters, a configuration, or a system behavior) that maximizes a latent user utility accessible only through costly, noisy, and indirect queries to the human.

$$x^* = \arg \max_{x \in X} u(x)$$

The best design maximizes the latent utility.

Summary of mathematical notations

$$x \in X$$

A Design in a Design Space

$$u : X \rightarrow \mathbb{R}$$

Unknown user utility function u takes any design x from the design space X and assigns it a single real number,

$$q \in Q$$

We need to query the user.

$$r \sim p(r|q; u)$$

How we only have access to a response r

$$c(q) > 0$$

And queries are not for free.

$$x^* = \arg \max_{x \in X} u(x)$$

The goal is then to find a design that maximizes this utility.

Write down possible definitions, values, or examples that a health dashboard could take, for each part of the definition.

$$x \in X$$

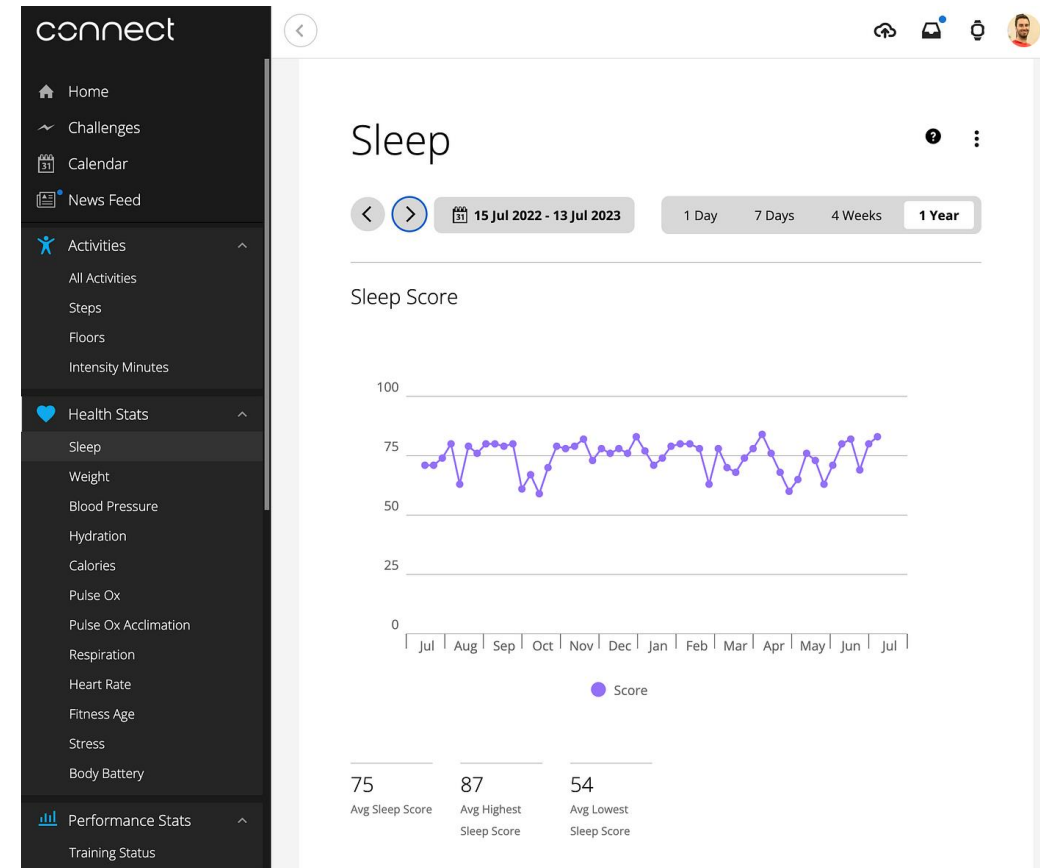
$$u : X \rightarrow \mathbb{R}$$

$$q \in Q$$

$$r \sim p(r|q; u)$$

$$c(q) > 0$$

$$x^* = \arg \max_{x \in X} u(x)$$



Imagine a Design Task

- X groups
- Come up with example domains – one topic per group will be discussed
- How would you approach the design?
 - What are the parameters that you would change for design?
 - What are your objectives for the design?
 - Which challenges you think you would encounter?

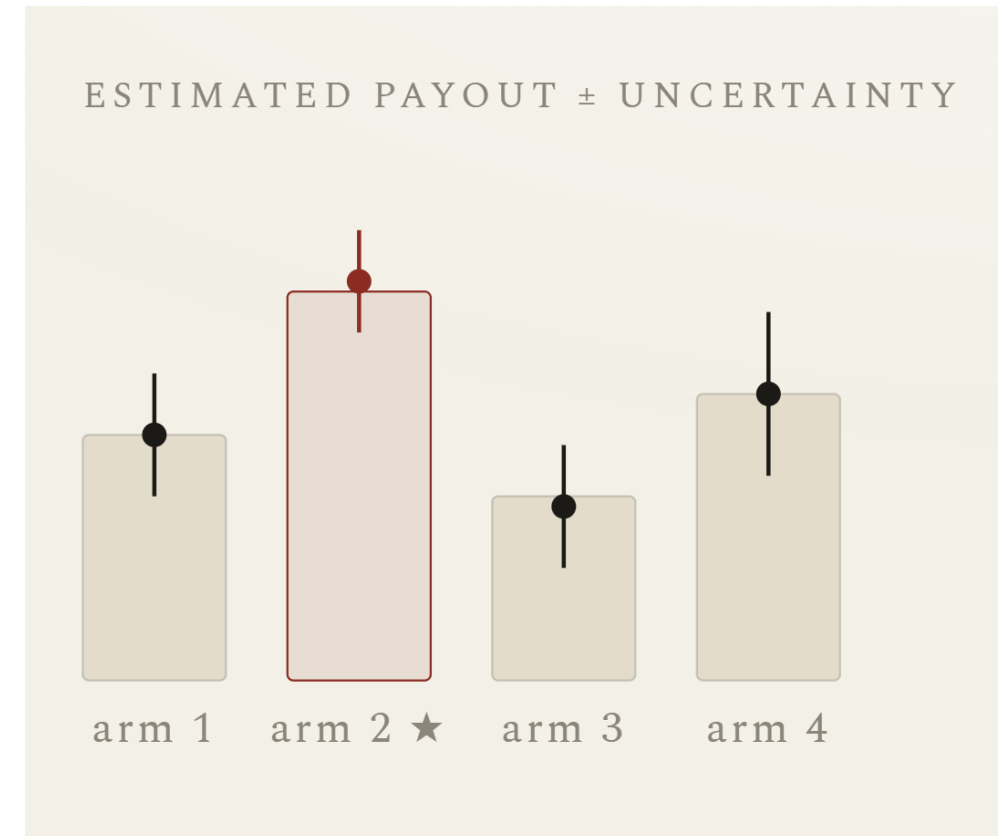
Methods

Multi-Armed Bandits

Intuition. A row of slot machines, each with an unknown payout. Every pull is a query; you want the most total reward while you learn which arm is best — the purest form of explore vs. exploit.

USE CASE

the options are few and discrete, you get a reward signal each round, and you care about performance *during* learning — not just at the end. A/B tests, recommendation, UI variants.

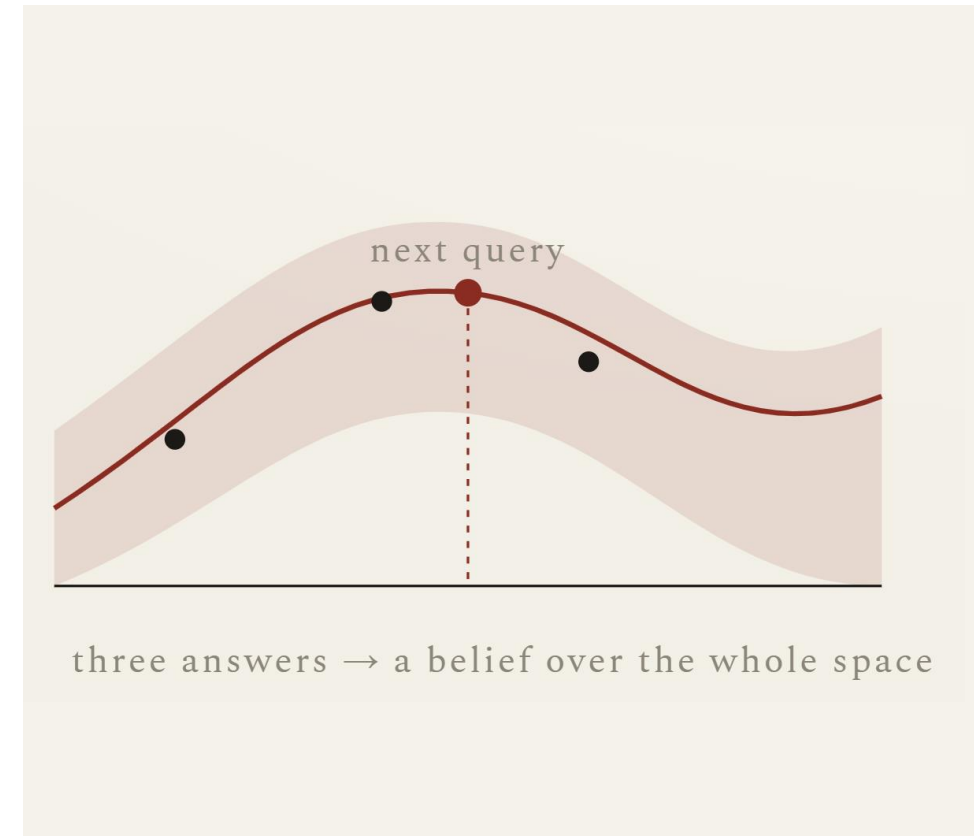


Bayesian Optimization

Intuition. Fit a smooth belief over the whole space from a few points — usually a Gaussian process — then let an acquisition function pick the single most informative place to ask next. Exactly the surrogate-plus-acquisition loop from Part III.

USE CASE

designs are a handful of continuous parameters, each evaluation is expensive, and you believe utility varies smoothly. Tuning, calibration, parameter design.

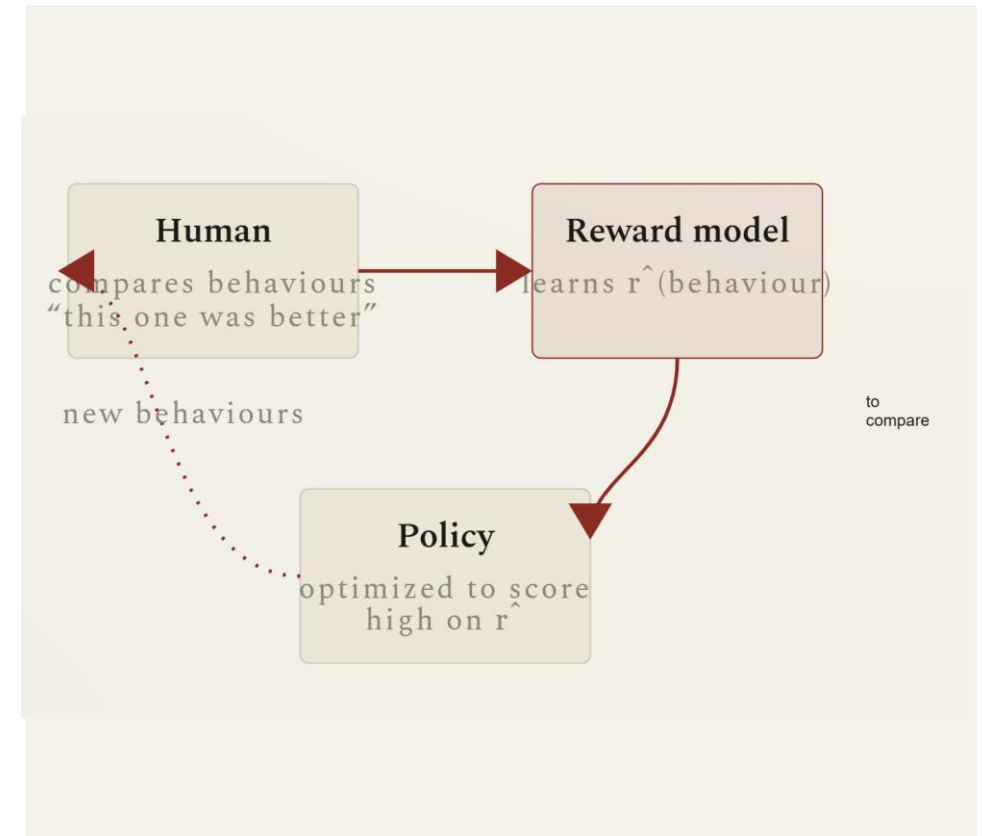


Reinforcement Learning from Human Feedback

Intuition. Sometimes x isn't a few sliders — it's a policy that acts over time. RLHF learns a **reward model** from human comparisons of behaviour, then optimizes a policy against it. Utility becomes a learned reward; the loop becomes reinforcement learning.

USE CASE

Tuning sequential behaviour, the space is enormous, and people can judge outcomes far more easily than specify them. Agents, dialogue, control.



The whole module in five lines:

1. So much of HCI is getting a system to behave the way a **user** wants — a choice of *design*.
2. “Good” means high *utility* $u(x)$ — but u is **latent**, locked in the user's head.
3. We can only *query* it, and answers are **noisy** and **costly**.
4. So we model a belief $\hat{u}$ and spend each query where it helps most — the *loop*.
5. Bandits, Bayesian Optimization, and RLHF are three ways to run that loop.

Follow-up Lectures

- Multi-Armed Bandits
- Bayesian Optimization: Basics
- Bayesian Optimization: Advanced
- Reinforcement Learning from Human Feedback

Related Modules

Design Optimization

How to optimize designs without the human in the loop.

Suggested Readings:

- Chan, L., Liao, Y. C., Mo, G. B., Dudley, J. J., Cheng, C. L., Kristensson, P. O., & Oulasvirta, A. (2022, April). Investigating positive and negative qualities of human-in-the-loop optimization for designing interaction techniques. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (pp. 1-14).